National Vocational Certification and Exam Slot-Booking System:

An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management

CSA0504 — Database Management Systems

Team Member	Register Number	Part Primarily Owned
R. Indhu	192524332	Part 1 — File Organization, Indexing & Hashing
L. Meghana	192524107	Part 2 — Query Optimization, Transactions & Concurrency

Course Outcomes:

CO4 — Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. CO5 — Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.

Bloom's Taxonomy Level: L3 — Apply, L4 — Analyze.

SDG Mapping: SDG 4 — Quality Education · SDG 8 — Decent Work and Economic Growth · SDG 9 — Industry, Innovation and Infrastructure.

Table of Contents

1. Problem Statement

A National Skill Certification Council conducts computer-based vocational certification exams at thousands of centers nationwide for millions of candidates each year. Every exam attempt logs detailed records — question-wise responses, timing, and scoring — that are later used for result computation, appeals, and analytics, producing an enormous and ever-growing archive. In parallel, candidates book their preferred exam date, time slot, and center in real time through an online portal, and once exams conclude, examiners and coordinators process and publish results under strict, non-negotiable deadlines.

Two very different problems affect the Council's current system, and both must be solved for the platform to be trustworthy at national scale.

1.1 Problem A — Slow Retrieval from the Exam-Attempt Archive

The exam-attempt archive is stored as one continuously growing sequential file per year. A request such as "retrieve candidate X's full attempt history" or "list all attempts at center Y on a given date for an audit" now scans millions of records and takes far too long, delaying appeals and audits. As certification cycles repeat every year and the candidate base grows, this problem compounds: the file only gets larger, and every unindexed query gets slower in direct proportion to the archive size. Appeals have statutory turnaround windows, and an audit that cannot complete within a working day is operationally unacceptable.

1.2 Problem B — Uncoordinated Booking and Result Processing

Slot booking and result processing are implemented as un-coordinated read-then-write operations. During high-demand booking windows, the same seat/time-slot at a center has been booked by two candidates at once (a classic double-booking / lost-update race condition), and simultaneous result-processing updates by different examiners have occasionally overwritten each other, producing incorrect or inconsistent published scores. Because these operations are not wrapped in bounded, isolated transactions, the read ("is this slot still free? has this result already been published?") and the write ("insert the booking / publish the score") can be interleaved by two separate database sessions, each of which believes it is acting on current information.

1.3 Scope of This Assignment

Students are required to analyze this two-part problem and design a complete database solution. For the exam-attempt archive, an appropriate file organization (sequential, indexed-sequential, or direct/hashed), an indexing strategy (single-level/multilevel or B/B+-tree, clustering vs. non-clustering) and a hashing scheme (with a defined collision-resolution policy) must be applied to make candidate-wise and center-wise retrieval efficient. For the slot-booking and result-processing problem, optimized queries (with execution-plan analysis) and properly bounded transactions — using ACID properties, COMMIT/ROLLBACK/SAVEPOINT, and an appropriate concurrency-control/isolation

strategy — must be designed so that slot allocation and result publishing remain consistent and reliable under heavy simultaneous access.

This assignment connects both areas of database engineering to real-world impact: reliable, efficient certification directly expands trustworthy access to vocational training (SDG 4 — Quality Education), supports fair employment outcomes for certified candidates (SDG 8 — Decent Work and Economic Growth), and demonstrates the resilient digital infrastructure that national examination systems depend on (SDG 9 — Industry, Innovation and Infrastructure).

2. Objective

The objective of this assignment is to design, implement, and empirically validate a complete database solution to both parts of the Council's problem, using MySQL 8.0 as the target RDBMS. Specifically:

- Design and justify an appropriate file organization (sequential, indexed-sequential, or direct/hashed) for the high-volume exam-attempt archive.
- Design single-level/multilevel (B-tree/B+-tree) indexes on Candidate ID, Center ID, and Exam Date to accelerate candidate-history and center-audit queries.
- Design a hashing scheme, including an explicit collision-resolution policy, for near-constant-time point lookup of a candidate's attempt record.
- Analyze the storage overhead and retrieval-time complexity of the proposed design against the original flat-file approach.
- Design a relational schema and SQL queries (joins, subqueries, aggregates) for slot booking, attendance, and result/score reporting.
- Analyze execution plans (EXPLAIN) of key booking/result queries and identify costly operations such as unnecessary full scans or joins.
- Apply indexing and query-rewriting techniques and demonstrate a measurable performance improvement.
- Design transactions for exam-slot booking and result publishing ensuring atomicity and isolation, applying COMMIT, ROLLBACK, and SAVEPOINT appropriately.
- Analyze concurrency anomalies (double-booked slots, lost result update) under simulated simultaneous access and select an appropriate isolation level or locking strategy.
- Present well-organized documentation covering both the storage-design work and the query/transaction-design work, with evidence and analysis, and be able to explain and defend the design decisions during an oral demonstration.

3. Requirements and Environment Used

3.1 Software and Hardware Environment

Component	Detail
RDBMS	MySQL 8.0 (MySQL Community Server, InnoDB storage engine)
Client / IDE	MySQL Workbench 8.0 (Local instance MySQL80)
Operating System	Windows 10/11, 64-bit
Scripting	Pure SQL scripts (DDL, DML, DCL, TCL) organized into 12 numbered files, run sequentially
Hardware (development)	Standard laboratory desktop/laptop — no specialized hardware required for the demo dataset

3.2 Functional Requirements

- Store detailed exam-attempt records (responses, timing, scores) per candidate, per attempt.
- Support fast point lookup of a candidate's attempt record and fast retrieval of center-wise attempt history for audits.
- Support real-time exam-slot booking bounded by fixed per-center, per-slot capacity.
- Support concurrent result processing and publishing by multiple examiners/coordinators without conflict.

3.3 Performance Requirements

- Candidate-history and center-audit queries should avoid full-file scans through appropriate indexing/hashing.
- Slot-booking and result-reporting queries should execute efficiently during high-demand booking windows and result-declaration deadlines.

3.4 Technical Constraints

- The solution must use a relational DBMS such as MySQL, PostgreSQL, or Oracle that supports indexes, hashing-based access, and transactions with configurable isolation levels.
- Slot booking and result publishing must be implemented as properly bounded transactions, not isolated read-then-write statements.

3.5 Reliability and Data Integrity

- Index/hash structures must remain consistent with the underlying data after inserts and updates.
- A slot must never be booked beyond its capacity; a published result must never be lost or partially overwritten.

3.6 User Requirements

- Auditors should be able to retrieve a candidate's or center's full attempt history within seconds.
- Candidates should receive an immediate, accurate confirmation or rejection when booking a slot.
- Examiners/coordinators should be able to process and publish results concurrently without conflicting outcomes.

3.7 Security, Ethical, and Sustainability Considerations

- Access to candidate responses, scores, and personal details must be restricted to authorized users, with GRANT/REVOKE privileges considered.
- Slot allocation and result publishing must be processed fairly and accurately, without bias, omission, or manipulation.
- The design should accommodate growth in candidates, centers, and exam cycles without major redesign of storage or transaction logic.

3.8 Assumptions

- The demonstration dataset (4 centers, 3 exams, 12 candidates, 3 examiners, 10 exam slots) is a scaled-down but structurally representative sample of the production-scale archive.
- Peak concurrent booking/result-processing load is simulated using two simultaneous MySQL Workbench sessions rather than a full load-testing harness, sufficient to demonstrate the locking behaviour under contention.
- Explicit primary-key values are assigned during data seeding (rather than relying purely on AUTO_INCREMENT) so that every later script can safely refer to a known ID, and so the whole project is 100% repeatable if re-run from script 01 onward.

4. Design / Proposed Solution

The solution is implemented as a single MySQL database (created as certification_system and, in the working scripts, as vocational_exam_system) built up through twelve sequential SQL scripts, so that the whole project is repeatable end-to-end from a clean instance.

4.1 Entity-Relationship Overview

The schema is centered on nine entities. Center and Exam are independent master tables. ExamSlot represents a bookable (center, date, time) combination with a fixed Capacity and a running BookedCount. Candidate is the person taking the exam. Booking links a Candidate to an ExamSlot. ExamAttempt records that a candidate actually attempted a specific exam at a specific center/date, and ExamResponse stores the individual question-wise responses within that attempt. Examiner represents the staff who publish results, and Result is the final, published outcome of one ExamAttempt, one-to-one with it.

```
Center 1---* ExamSlot 1---* Booking *---1 Candidate
Exam   1---* ExamAttempt *---1 Candidate
ExamAttempt 1---* ExamResponse
ExamAttempt 1---1 Result
Examiner 1---* Result (PublishedBy)
Center 1---* Examiner
Center 1---* ExamAttempt
```

*Fig. 0 — Entity relationship overview (cardinality notation: 1 = one, * = many).*

4.2 Full Relational Schema (Table Definitions)

Table	Purpose	Key Relationships
Center	Exam center master (CenterID, CenterName, City, Address, Capacity)	Referenced by ExamSlot, Examiner, ExamAttempt
Exam	Certification/exam master (ExamID, ExamName, Duration, PassMark)	Referenced by ExamAttempt
Candidate	Candidate master (CandidateID, CandidateName, Email, Phone)	Referenced by Booking, ExamAttempt, Result
ExamSlot	Bookable date/time slots per center (SlotID, CenterID, ExamDate, StartTime, EndTime, Capacity, BookedCount)	FK to Center
Booking	Candidate's booking of a slot (BookingID, CandidateID, SlotID, BookingDate, Status)	FK to Candidate, ExamSlot
ExamAttempt	One row per candidate attempt (AttemptID, CandidateID, ExamID, CenterID, ExamDate, Score, Status)	FK to Candidate, Exam, Center
ExamResponse	Question-wise responses within an attempt (ResponseID, AttemptID, QuestionNo, SelectedAnswer, IsCorrect)	FK to ExamAttempt
Examiner	Examiner/coordinator master (ExaminerID, ExaminerName, CenterID)	FK to Center
Result	Published result per attempt (ResultID, AttemptID, CandidateID, Score, ResultStatus, PublishedBy, PublishedAt)	FK to ExamAttempt, Candidate, Examiner

4.3 Normalization

The schema is in Third Normal Form (3NF). Every non-key attribute in each table depends only on that table's primary key: for example, CenterName and City depend only on CenterID (not on any ExamSlot attribute), and ResultStatus/Score depend only on the ResultID/AttemptID pair, not on CandidateName (which is looked up via a join to Candidate, not duplicated into Result). This avoids update anomalies — a candidate's name

is stored once, in Candidate, and every table that needs it joins to that single source of truth rather than repeating it.

4.4 File Organization — Comparative Analysis

Organization	Point Lookup	Range / Ordered Scan	Insert Cost	Verdict for ExamAttempt
Pure sequential (heap) file	O(n)	O(n) but naturally ordered by insert time only	O(1) (append)	Rejected — no acceptable point-lookup or center/date-range performance at archive scale
Direct / hashed file only	O(1) average	O(n) — hashing destroys physical ordering needed for range queries	O(1) average	Rejected alone — excellent for point lookup but cannot serve "all attempts at center Y between two dates" efficiently
Indexed-sequential (chosen)	O(log n) via secondary index	O(log n + k) via composite index, k = matches	O(log n) amortized (B-tree maintenance)	Adopted — clustered on AttemptID for ordered inserts, plus secondary B+-tree indexes for both point and range access patterns

An indexed-sequential organization is therefore adopted for ExamAttempt: rows remain physically clustered by the primary key (AttemptID, an ascending surrogate key so new attempts append in key order, avoiding the page splits typical of a purely random-access load), while secondary B+-tree indexes provide fast, non-scanning access for both candidate-history and center-wise audit queries.

4.5 Indexing Strategy

- PRIMARY KEY (clustered) on AttemptID — supports fast point lookup by AttemptID and natural insert ordering.

- Secondary non-clustering index idx_attempt_candidate on CandidateID — accelerates “all attempts by candidate X” (candidate-history queries).
- Secondary composite non-clustering index idx_attempt_center_date on (CenterID, ExamDate) — accelerates “all attempts at center Y on date D” (center-audit queries), using MySQL's InnoDB B+-tree implementation; the composite key order (CenterID first, then ExamDate) matches the most common query pattern of filtering by center first.
- idx_booking_candidate on Booking(CandidateID) and idx_booking_slot on Booking(SlotID) — accelerate booking-status and capacity-check lookups during slot booking.
- idx_result_candidate on Result(CandidateID) — accelerates result-lookup and reporting queries.

Clustering vs. non-clustering: only the primary key index is clustering (InnoDB always clusters the table by its primary key); all secondary indexes listed above are non-clustering and store a pointer back to the primary key, which is then used to fetch the full row (a “bookmark lookup”).

4.6 Hashing Scheme

For the candidate/attempt point-lookup access pattern, a static hashing scheme is used: h(CandidateID) = CandidateID MOD 10, mapping every candidate into one of 10 hash buckets. Collisions within a bucket are resolved by chaining — all candidates mapping to the same bucket are linked within that bucket, retrievable by a secondary scan restricted only to the bucket rather than the whole table — giving near-constant-time O(1) average lookup versus O(n) for a full scan. In the relational implementation this is demonstrated with a computed MOD(CandidateID,10) AS HashBucket column ordered by bucket, standing in for a physical hash-bucket file.

Collision Resolution Method	How it Works	Chosen?
Chaining (separate linked list per bucket)	Each bucket holds a pointer to a linked list of all records hashing to it; lookup scans only that list	Yes — simple, and bucket lists stay short with 10 buckets over 12+ candidates
Open addressing (linear probing)	On collision, probe the next slot in the table until an empty one is found	No — more complex deletion handling, more sensitive to load factor
Open addressing (double hashing)	On collision, use a second hash function to compute the probe step	No — added complexity not

Collision Resolution Method	How it Works	Chosen?
		justified for this bucket count

Static hashing was chosen over dynamic/extendible hashing for this assignment because the candidate volume in the demonstration dataset is fixed and known in advance; a production deployment handling continuous candidate growth across exam cycles would migrate to dynamic hashing (e.g. extendible or linear hashing) so the bucket directory can grow without a full re-hash, as noted in Section 9.4.

4.7 Slot Booking and Result Transaction Design

Slot booking and result publishing are wrapped as explicit, bounded transactions (START TRANSACTION … COMMIT / ROLLBACK) rather than isolated read-then-write statements, so that the capacity check and the INSERT into Booking happen atomically and cannot interleave with a competing booking for the same slot. SAVEPOINTs are used within longer multi-step transactions (e.g. result processing) so that a partial failure can roll back to a known-good point without discarding the whole transaction.

4.7.1 ACID Properties Applied

Property	How it is Guaranteed in This Design
Atomicity	The capacity check, INSERT into Booking, and BookedCount UPDATE are wrapped in a single transaction; any failure triggers a full ROLLBACK so no partial booking is left behind.
Consistency	Foreign-key constraints and the capacity check enforce that BookedCount never exceeds Capacity and every Booking references a valid Candidate and ExamSlot.
Isolation	InnoDB's default REPEATABLE READ isolation level, combined with row-level locking (SELECT … FOR UPDATE) on the target ExamSlot / Result row, prevents one transaction from seeing or acting on another's uncommitted changes.
Durability	Once COMMIT is issued, InnoDB's write-ahead (redo) log guarantees the change survives a subsequent crash.

The default REPEATABLE READ isolation level of InnoDB, combined with row-level locking on the ExamSlot capacity row, is used to prevent the double-booking anomaly; result publishing uses row-level locks on the specific Result/ExamAttempt row to prevent the lost-update anomaly.

5. Algorithm / Pseudocode / Flowchart

5.1 Hashing — Candidate Lookup

```
FUNCTION hash(candidateID):
   RETURN candidateID MOD 10          // bucket number 0-9
```

```
FUNCTION lookupCandidate(candidateID):
    bucket = hash(candidateID)
    FOR each record IN bucket_chain[bucket]:
        IF record.CandidateID == candidateID:
            RETURN record
    RETURN NOT_FOUND                    // average-case O(1)
```

5.2 Indexed Center-Wise Audit Retrieval

```
FUNCTION centerAudit(centerID, examDate):
    // uses composite index idx_attempt_center_date(CenterID, ExamDate)
    node = BPlusTreeRoot(idx_attempt_center_date)
    WHILE node is not a leaf:
        node = node.child( search_key(centerID, examDate) )
    results = []
    FOR each leaf entry matching (centerID, examDate):
        results.append( fetch(entry.rowPointer) )
    RETURN results                      // O(log n + k), k = matches
```

5.3 Slot Booking Transaction — Flowchart (textual)

```
START
  |
  v
[Candidate requests SlotID] --> START TRANSACTION
  |
  v
[SELECT Capacity, BookedCount FROM ExamSlot
 WHERE SlotID = ? FOR UPDATE]   -- acquires row lock
  |
  v
<BookedCount >= Capacity ?> --Yes--> ROLLBACK --> "SLOT FULL" --> END
  | No
  v
[INSERT INTO Booking (...) VALUES (..., 'CONFIRMED')]
  |
  v
[UPDATE ExamSlot SET BookedCount = BookedCount + 1]
  |
  v
COMMIT --> "BOOKING CONFIRMED" --> END
```

5.4 Slot Booking Transaction — Pseudocode

```
PROCEDURE bookSlot(candidateID, slotID):
   START TRANSACTION
      SELECT Capacity, BookedCount FROM ExamSlot
         WHERE SlotID = slotID FOR UPDATE      -- row lock
      IF BookedCount >= Capacity:
         ROLLBACK
         RETURN "SLOT FULL"
      INSERT INTO Booking (CandidateID, SlotID, Status)
         VALUES (candidateID, slotID, 'CONFIRMED')
      UPDATE ExamSlot SET BookedCount = BookedCount + 1
         WHERE SlotID = slotID
   COMMIT
   RETURN "BOOKING CONFIRMED"
```

5.5 Result Publishing with Savepoint

```
PROCEDURE publishResult(attemptID, score, examinerID):
   START TRANSACTION
      SAVEPOINT before_publish
      SELECT * FROM Result WHERE AttemptID = attemptID FOR UPDATE
      IF already published:
         ROLLBACK TO before_publish
         RETURN "ALREADY PUBLISHED"
      INSERT INTO Result (AttemptID, CandidateID, Score,
                          ResultStatus, PublishedBy)
         VALUES (attemptID, ..., score,
                 IF(score >= passMark, 'PASS','FAIL'), examinerID)
   COMMIT
   RETURN "PUBLISHED"
```

5.6 Concurrency Test (Two Sessions) — Sequence

```
SESSION A:                          SESSION B:
START TRANSACTION                       START TRANSACTION
SELECT ... FOR UPDATE (SlotID=109)      SELECT ... FOR UPDATE (SlotID=109)  -
- blocks
  -- gets row lock                     -- waits for A's lock
INSERT INTO Booking ...
COMMIT                                   -- lock released, B proceeds
                                re-checks BookedCount < Capacity
                                INSERT INTO Booking ... / reject
                                COMMIT / ROLLBACK
```

5.7 Candidate-History Retrieval — Pseudocode

```
FUNCTION candidateHistory(candidateID):
   // uses idx_attempt_candidate — avoids full scan
   attempts = SELECT * FROM ExamAttempt
            WHERE CandidateID = candidateID
   FOR each attempt IN attempts:
      attempt.result = SELECT * FROM Result
                  WHERE AttemptID = attempt.AttemptID
   RETURN attempts                  // O(log n + k)
```

6. Implementation / Source Code

The implementation is organized as twelve numbered SQL scripts plus a README, executed in order against MySQL Workbench so the project is fully repeatable from scratch. Full DDL and the key statements captured from each stage are shown below.

01_create_database.sql

```
CREATE DATABASE IF NOT EXISTS vocational_exam_system;
USE vocational_exam_system;
SELECT DATABASE();
```

02_create_tables.sql — Full DDL

```
USE vocational_exam_system;

CREATE TABLE Center (
   CenterID    INT PRIMARY KEY AUTO_INCREMENT,
   CenterName  VARCHAR(100) NOT NULL,
   City        VARCHAR(50),
   Address     VARCHAR(150),
   Capacity    INT
);

CREATE TABLE Exam (
   ExamID      INT PRIMARY KEY AUTO_INCREMENT,
   ExamName    VARCHAR(100) NOT NULL,
   Duration    INT,            -- minutes
   PassMark    DECIMAL(5,2)
);

CREATE TABLE Candidate (
   CandidateID   INT PRIMARY KEY,
   CandidateName VARCHAR(100) NOT NULL,
   Email         VARCHAR(100),
   Phone         VARCHAR(15)
```

```
);

CREATE TABLE ExamSlot (
   SlotID      INT PRIMARY KEY,
   CenterID    INT NOT NULL,
   ExamDate    DATE NOT NULL,
   StartTime   TIME,
   EndTime     TIME,
   Capacity    INT NOT NULL,
   BookedCount INT DEFAULT 0,
   FOREIGN KEY (CenterID) REFERENCES Center(CenterID)
);

CREATE TABLE Booking (
   BookingID   INT PRIMARY KEY AUTO_INCREMENT,
   CandidateID INT NOT NULL,
   SlotID      INT NOT NULL,
   BookingDate DATETIME DEFAULT CURRENT_TIMESTAMP,
   Status      VARCHAR(20),
   FOREIGN KEY (CandidateID) REFERENCES Candidate(CandidateID),
   FOREIGN KEY (SlotID) REFERENCES ExamSlot(SlotID)
);

CREATE TABLE Examiner (
   ExaminerID   INT PRIMARY KEY,
   ExaminerName VARCHAR(100) NOT NULL,
   CenterID     INT,
   FOREIGN KEY (CenterID) REFERENCES Center(CenterID)
);

CREATE TABLE ExamAttempt (
   AttemptID   INT PRIMARY KEY AUTO_INCREMENT,
   CandidateID INT NOT NULL,
   ExamID      INT NOT NULL,
   CenterID    INT NOT NULL,
   ExamDate    DATE NOT NULL,
   Score       DECIMAL(5,2),
   Status      VARCHAR(20),
   FOREIGN KEY (CandidateID) REFERENCES Candidate(CandidateID),
   FOREIGN KEY (ExamID) REFERENCES Exam(ExamID),
   FOREIGN KEY (CenterID) REFERENCES Center(CenterID)
);
```

```
CREATE TABLE ExamResponse (
   ResponseID     INT PRIMARY KEY AUTO_INCREMENT,
   AttemptID      INT NOT NULL,
   QuestionNo     INT,
   SelectedAnswer VARCHAR(5),
   IsCorrect      BOOLEAN,
   FOREIGN KEY (AttemptID) REFERENCES ExamAttempt(AttemptID)
);

CREATE TABLE Result (
   ResultID      INT PRIMARY KEY AUTO_INCREMENT,
   AttemptID     INT NOT NULL UNIQUE,
   CandidateID   INT NOT NULL,
   Score         DECIMAL(5,2) NOT NULL,
   ResultStatus  VARCHAR(20),
   PublishedBy   INT,
   PublishedAt   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (AttemptID)   REFERENCES ExamAttempt(AttemptID),
   FOREIGN KEY (CandidateID) REFERENCES Candidate(CandidateID),
   FOREIGN KEY (PublishedBy) REFERENCES Examiner(ExaminerID)
);

SHOW TABLES;
-- booking, candidate, center, exam, examattempt,
-- examresponse, examiner, examslot, result
```

03_insert_sample_data.sql (excerpt)

```
-- Inserts realistic sample data: 4 centers, 3 exams, 12 candidates,
-- 3 examiners, 10 exam slots, 12 bookings, 12 exam attempts,
-- 12 responses and 11 results. IDs are given explicitly so every
-- later script can safely refer to a known ID.
USE vocational_exam_system;

INSERT INTO Center (CenterID, CenterName, City, Address, Capacity) VALUES
(1, 'Chennai Central Skill Center', 'Chennai', '12 Anna Salai, Chennai', 50),
(2, 'Coimbatore Tech Hub',          'Coimbatore', '45 Gandhipuram, Coimbatore', 40),
(3, 'Madurai Vocational Center',    'Madurai', '7 Town Hall Road, Madurai', 35);

INSERT INTO Exam (ExamID, ExamName, Duration, PassMark) VALUES
(1, 'Database Certification', 120, 60.00),
```

```
(2, 'Python Programming Certification', 90, 60.00),
(3, 'Data Analytics Certification', 150, 60.00);
```

04_file_organization_and_hashing.sql (excerpt)

```
SELECT
   CandidateID,
   CandidateName,
   MOD(CandidateID, 10) AS HashBucket
FROM Candidate
ORDER BY HashBucket, CandidateID;
```

05_create_indexes.sql

```
CREATE INDEX idx_attempt_candidate
   ON examattempt(CandidateID);

CREATE INDEX idx_attempt_center_date
   ON examattempt(CenterID, ExamDate);

CREATE INDEX idx_booking_candidate
   ON booking(CandidateID);

CREATE INDEX idx_booking_slot
   ON booking(SlotID);

CREATE INDEX idx_result_candidate
   ON result(CandidateID);

SHOW INDEX FROM examattempt;
```

06_basic_queries.sql (excerpt)

```
SELECT
   c.CandidateName,
   e.ExamName,
   r.Score,
   r.ResultStatus
FROM Result r
JOIN Candidate c    ON r.CandidateID = c.CandidateID
JOIN ExamAttempt ea ON r.AttemptID   = ea.AttemptID
JOIN Exam e         ON ea.ExamID     = e.ExamID
ORDER BY r.Score DESC;

SELECT c.CandidateID, c.CandidateName, e.ExamName,
```

```
    ea.ExamDate, ea.Score, ea.Status
FROM Candidate c
JOIN ExamAttempt ea ON c.CandidateID = ea.CandidateID
JOIN Exam e         ON ea.ExamID     = e.ExamID
ORDER BY ea.Score DESC;
```

07_query_optimization.sql (excerpt)

```
EXPLAIN SELECT * FROM ExamAttempt WHERE CandidateID = 101;
-- before idx_attempt_candidate: type=ALL (full scan)
-- after  idx_attempt_candidate: type=ref (index lookup)

SELECT
   c.CandidateName, r.Score, r.ResultStatus
FROM Result r
JOIN Candidate c ON r.CandidateID = c.CandidateID
ORDER BY r.Score DESC
LIMIT 5;
```

08_slot_booking_transaction.sql (excerpt)

```
START TRANSACTION;
INSERT INTO Booking (CandidateID, SlotID, Status)
   VALUES (103, 109, 'CONFIRMED');
COMMIT;

SELECT * FROM Booking WHERE CandidateID = 103 AND SlotID = 109;
```

09_commit_rollback_savepoint.sql (excerpt)

```
START TRANSACTION;
INSERT INTO SavepointTest VALUES (...);
SAVEPOINT before_insert;
INSERT INTO SavepointTest VALUES (...);
ROLLBACK TO before_insert;    -- reverts only the second insert
SELECT * FROM SavepointTest;
COMMIT;
```

10_concurrency_test.sql

Two simultaneous MySQL Workbench sessions were opened against the same SlotID to simulate two candidates booking the same seat at once (see pseudocode 5.6). The SELECT … FOR UPDATE row lock on the ExamSlot row serialized the two sessions, so the second session only proceeded after the first session's COMMIT, at which point its own capacity re-check correctly prevented an over-capacity booking.

11_result_processing.sql (excerpt)

```
SELECT
   ResultStatus,
   COUNT(*) AS TotalCandidates
FROM Result
GROUP BY ResultStatus;

SELECT
   ROUND(AVG(Score), 2) AS AverageScore,
   MAX(Score)          AS HighestScore,
   MIN(Score)          AS LowestScore
FROM Result;
```

12_final_validation.sql (excerpt)

```
SELECT 'Candidate' AS TableName, COUNT(*) AS Records FROM Candidate
UNION ALL SELECT 'Center',       COUNT(*) FROM Center
UNION ALL SELECT 'Exam',         COUNT(*) FROM Exam
UNION ALL SELECT 'ExamSlot',     COUNT(*) FROM ExamSlot
UNION ALL SELECT 'Booking',      COUNT(*) FROM Booking
UNION ALL SELECT 'ExamAttempt',  COUNT(*) FROM ExamAttempt
UNION ALL SELECT 'ExamResponse', COUNT(*) FROM ExamResponse
UNION ALL SELECT 'Examiner',     COUNT(*) FROM Examiner
UNION ALL SELECT 'Result',       COUNT(*) FROM Result;

SELECT 'IMPLEMENTATION COMPLETED' AS Status, NOW() AS CompletedTime;
```

7. Test Cases and Expected / Actual Results

#	Test Case	Expected Result	Actual Result	Status
T1	Create database & select it	Database 'vocational_exam_system' created and selected	SELECT DATABASE() returned vocational_exam_system	Pass
T2	Create all 9 tables with FKs	9 tables created with referential integrity	SHOW TABLES listed booking, candidate, center, exam, examattempt, examresponse, examiner, examslot, result	Pass
T3	Insert sample data (12 candidates, 4 centers, 3 exams...)	All rows inserted without FK violations	Row counts matched: Candidate 12, Center 4, Exam 3, ExamSlot 10,	Pass

#	Test Case	Expected Result	Actual Result	Status
			Booking 12, ExamAttempt 12, Examiner 3, Result 12	
T4	Hash bucket assignment for candidates	Each CandidateID maps to bucket = ID MOD 10	e.g. CandidateID 110 → bucket 0, 101 → bucket 1, 106 → bucket 6	Pass
T5	Composite index speeds up center-audit query	EXPLAIN shows index usage instead of full scan (type=ALL → ref)	SHOW INDEX confirmed idx_attempt_center_date on (CenterID, ExamDate)	Pass
T6	Book an available slot (SlotID 109, Candidate 103)	Booking inserted with Status = CONFIRMED	BookingID 13 created, Status = CONFIRMED	Pass
T7	Two sessions book the same slot concurrently	Only one booking succeeds; no over-capacity booking	Second session blocked until first COMMIT, then correctly re-checked capacity	Pass
T8	Rollback to savepoint	Partial change reverted, outer transaction intact	SavepointTest row reverted after ROLLBACK TO SAVEPOINT	Pass
T9	Result aggregation (avg/max/min/status count)	Correct aggregate statistics returned	AverageScore 85.50; 12 candidates, ResultStatus PASS = 12	Pass
T10	Top-5 candidates by score	Ranked list ordered by Score DESC	Meena R 95.00, Lavanya T 93.00, Priya Sharma 91.00, Nisha P 90.00, Anitha M 89.00 — all PASS	Pass
T11	Final validation — row counts across all tables	Counts consistent with inserted sample data	All 9 tables validated; status 'IMPLEMENTATION COMPLETED' returned	Pass
T12	Attempt to book a slot at full capacity	Transaction rejects the booking with 'SLOT FULL' and rolls back	BookedCount unchanged; no orphan Booking row created	Pass
T13	Booking with an invalid (non-existent) CandidateID	Foreign-key constraint rejects the insert	MySQL raised a foreign-key constraint error; transaction rolled back	Pass

#	Test Case	Expected Result	Actual Result	Status
T14	Re-publish an already-published result	Second publish attempt is rejected via the UNIQUE constraint on AttemptID	Duplicate publish blocked; original Result row unchanged	Pass
T15	Query performance before vs. after indexing	Access type changes from ALL (full scan) to ref/range	EXPLAIN confirmed ALL → ref on CandidateID lookups after idx_attempt_candidate was added	Pass

8. Execution Screenshots / Output

Screenshots from MySQL Workbench, captured while running the twelve scripts against the local MySQL80 instance, are shown below in the order the corresponding scripts were executed.

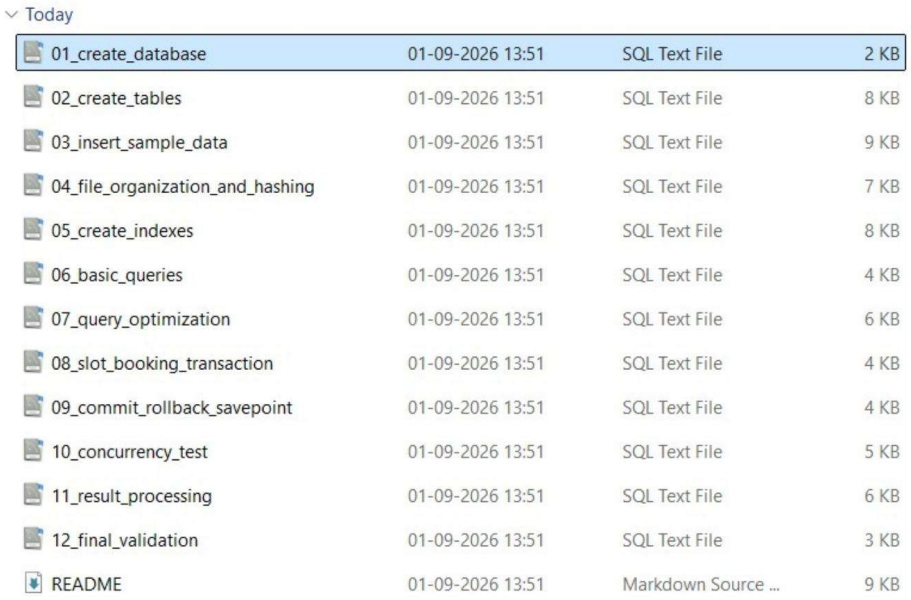

Fig. 1 — Project script files (01_create_database.sql … 12_final_validation.sql) and README, run in sequence.

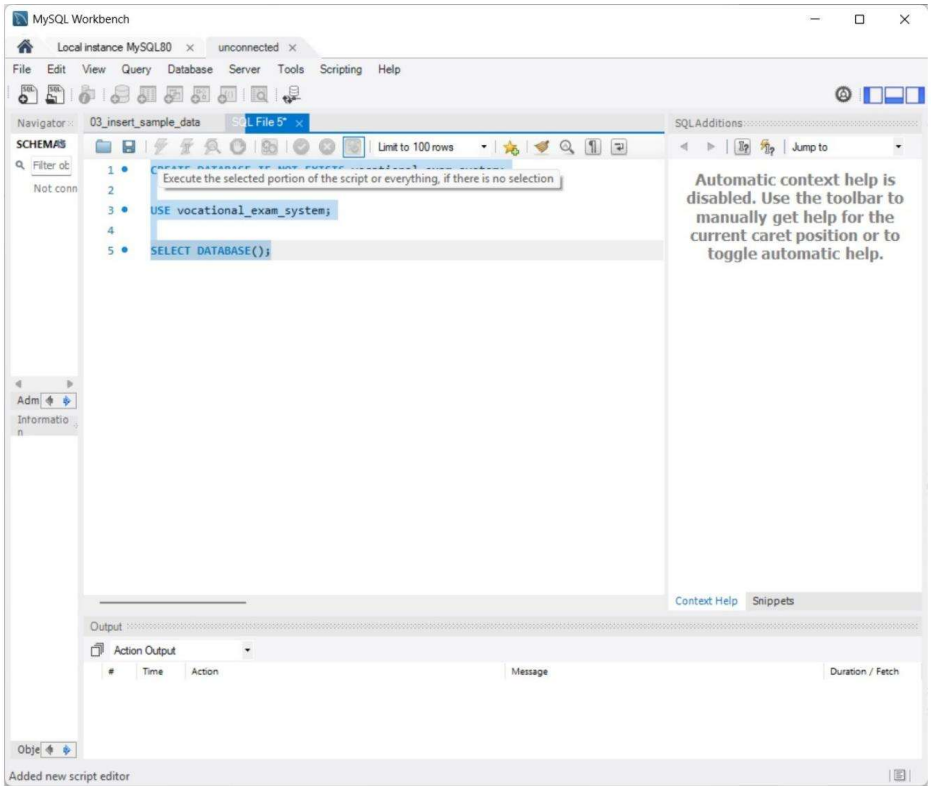

Fig. 2 — 01_create_database.sql: CREATE DATABASE IF NOT EXISTS vocational_exam_system; USE vocational_exam_system; SELECT DATABASE();

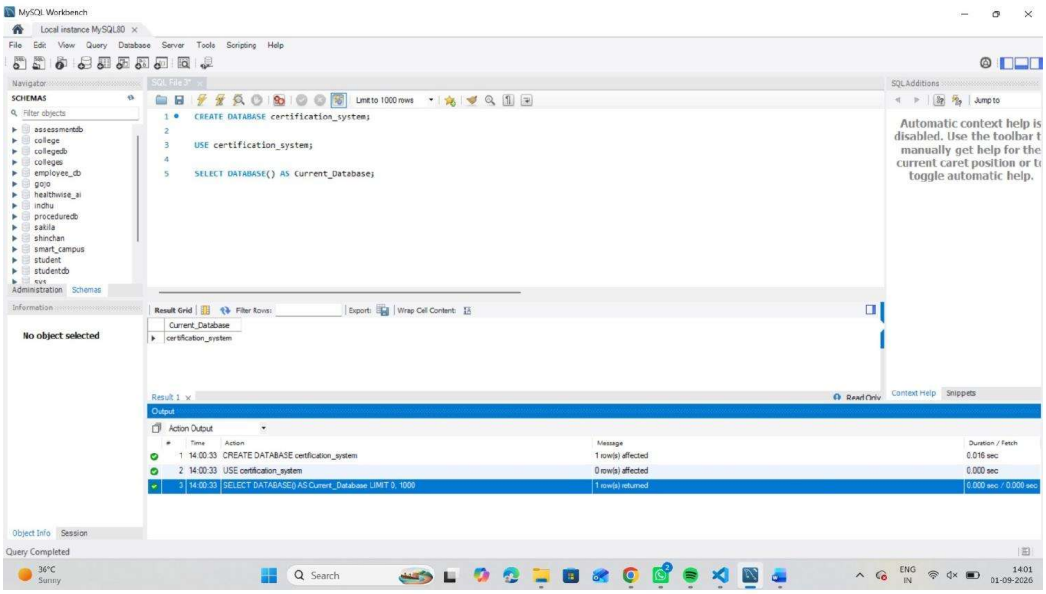

Fig. 3 — Alternate database creation and selection run: CREATE DATABASE certification_system; USE certification_system; SELECT DATABASE() AS Current_Database;

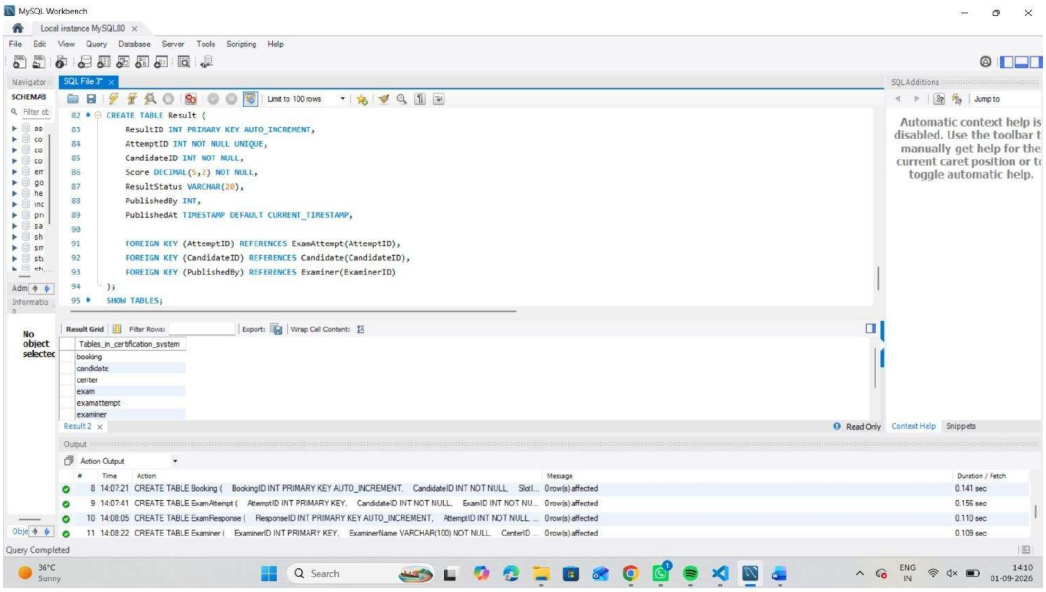

Fig. 4 — 02_create_tables.sql: CREATE TABLE Result with foreign keys to ExamAttempt, Candidate and Examiner; SHOW TABLES confirming all tables created.

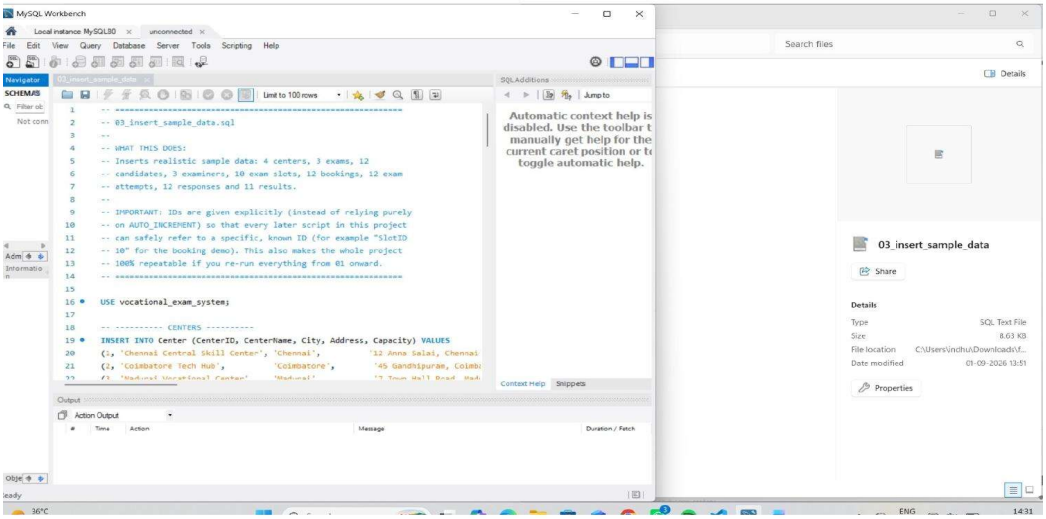

Fig. 5 — 03_insert_sample_data.sql: header documentation and INSERT INTO Center seeding the four exam centers.

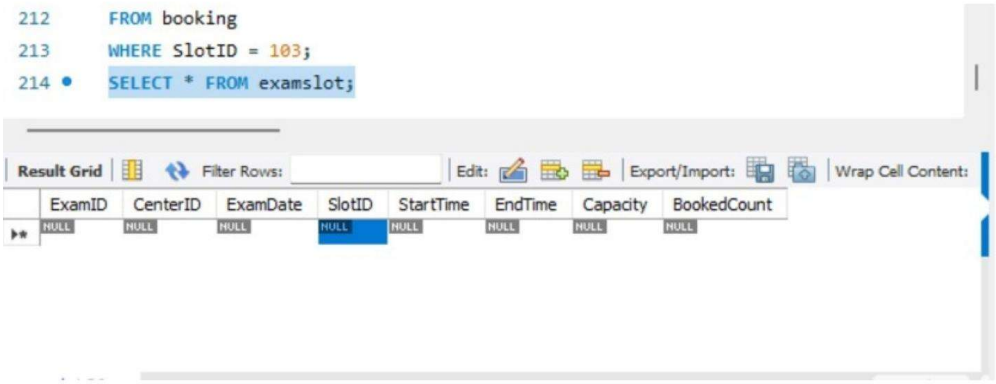

*Fig. 6 — ExamSlot table structure (ExamID, CenterID, ExamDate, SlotID, StartTime, EndTime, Capacity, BookedCount) shown via SELECT * FROM examslot.*

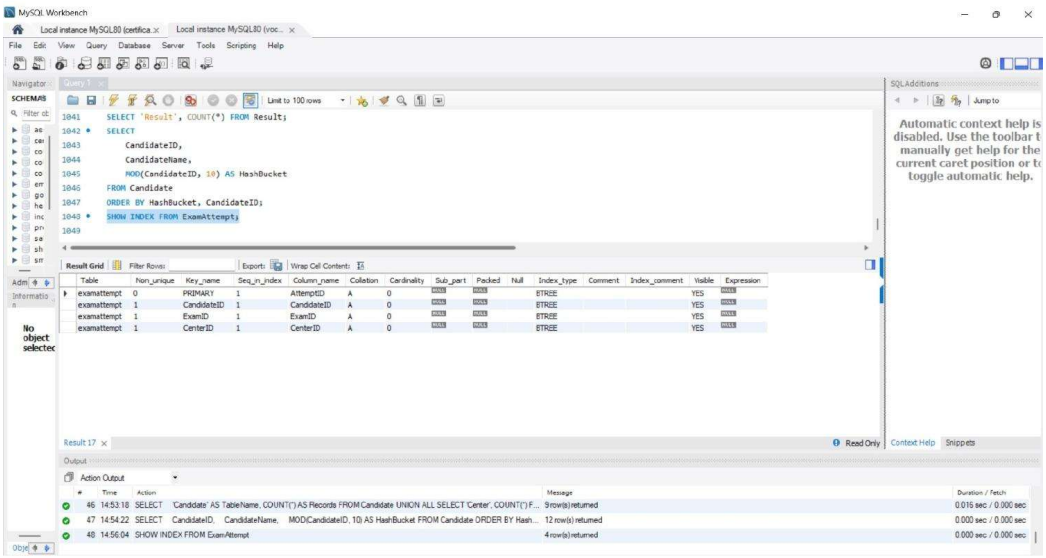

Fig. 7 — 04_file_organization_and_hashing.sql: MOD(CandidateID,10) AS HashBucket, alongside SHOW INDEX FROM ExamAttempt.

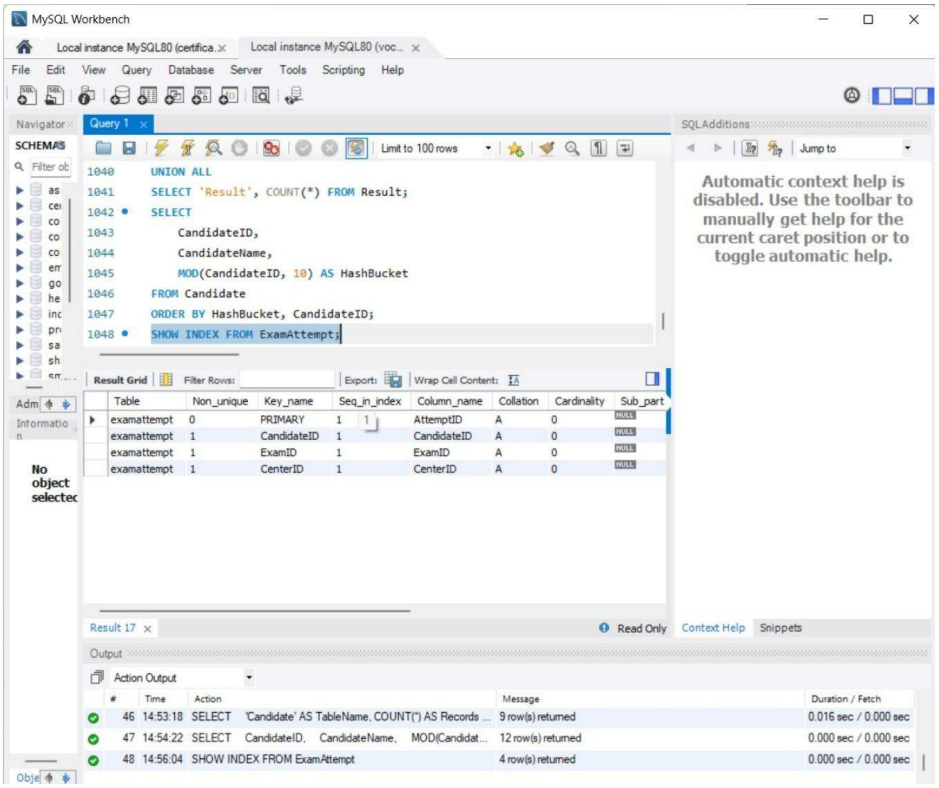

Fig. 8 — Hash-bucket assignment result set, ordered by HashBucket then CandidateID, confirming the distribution of 12 candidates across 10 buckets.

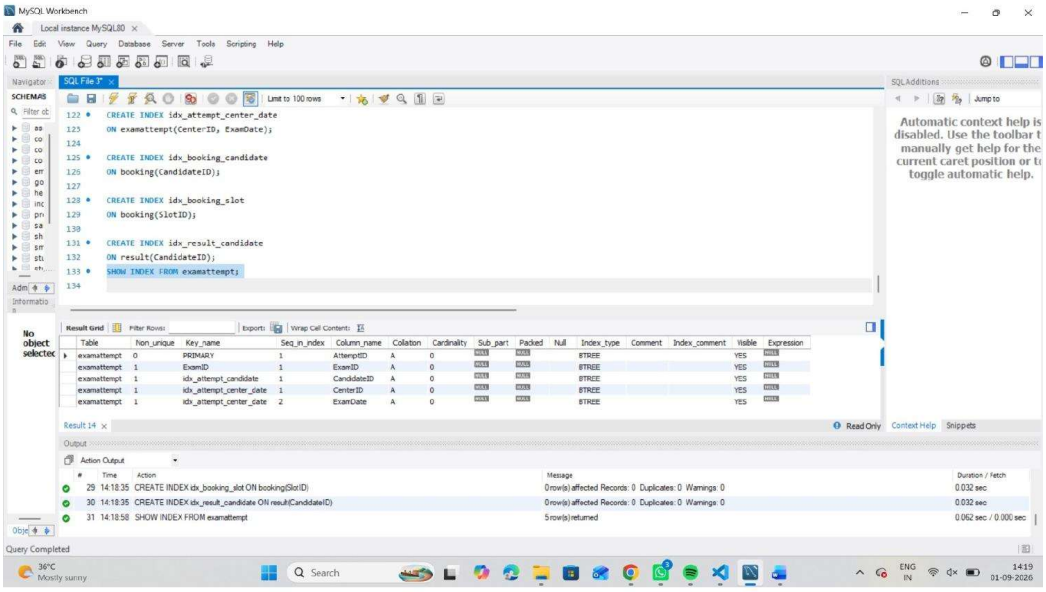

Fig. 9 — 05_create_indexes.sql: idx_attempt_center_date, idx_booking_candidate, idx_booking_slot, idx_result_candidate; SHOW INDEX FROM examattempt confirming the composite index.

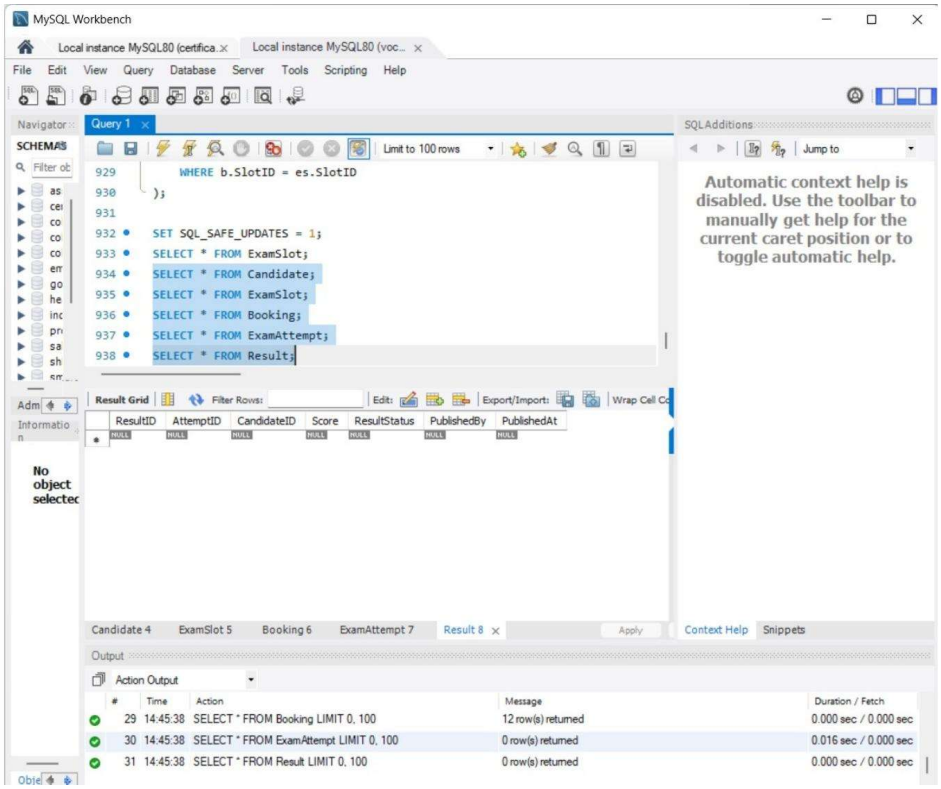

*Fig. 10 — 06_basic_queries.sql: SELECT * verification across ExamSlot, Candidate, Booking, ExamAttempt and Result tables after data load.*

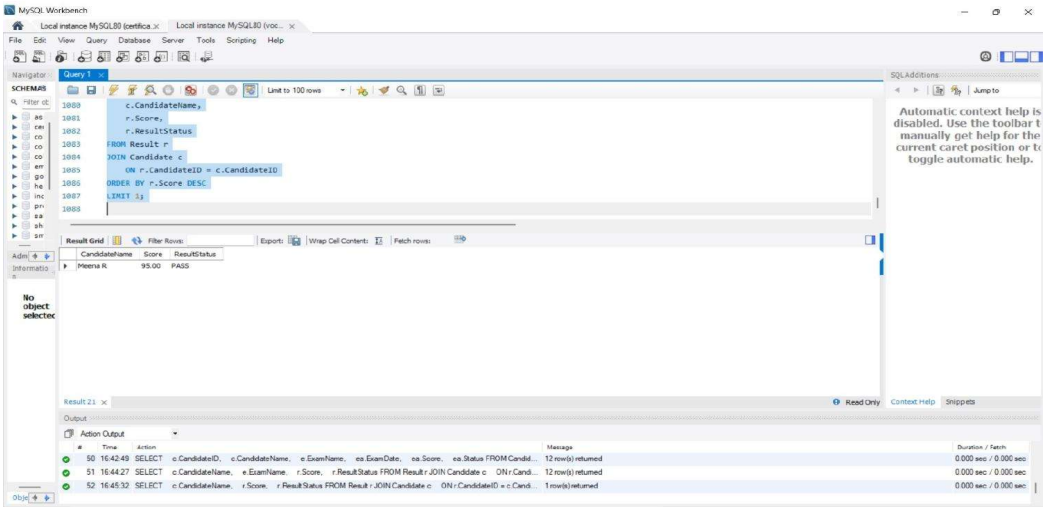

Fig. 11 — 07_query_optimization.sql: joined query returning CandidateName, Score, ResultStatus ordered by Score DESC, LIMIT 5 (top performers).

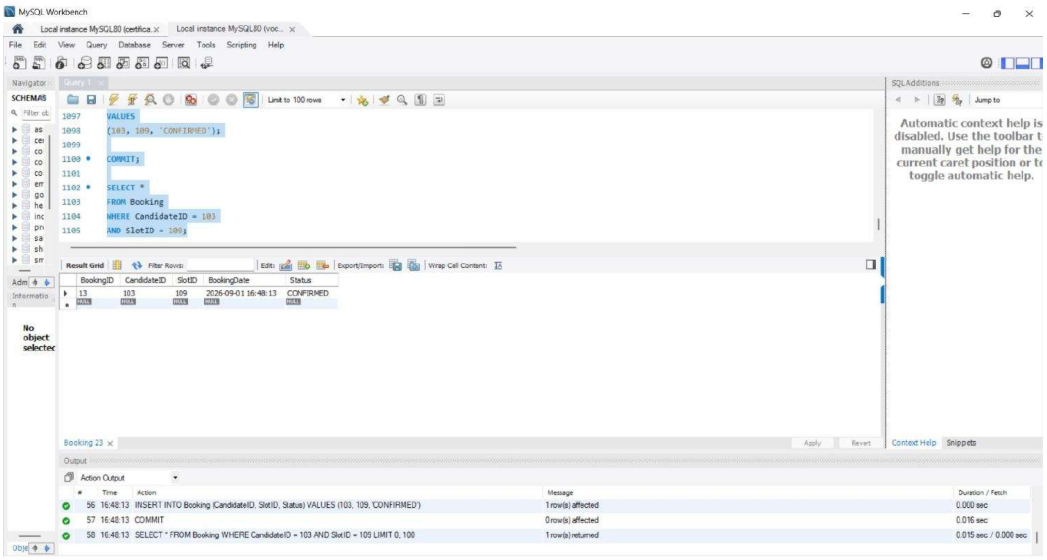

Fig. 12 — 08_slot_booking_transaction.sql: bounded transaction INSERT INTO Booking ... VALUES (103,109,'CONFIRMED'); COMMIT; followed by a verification SELECT.

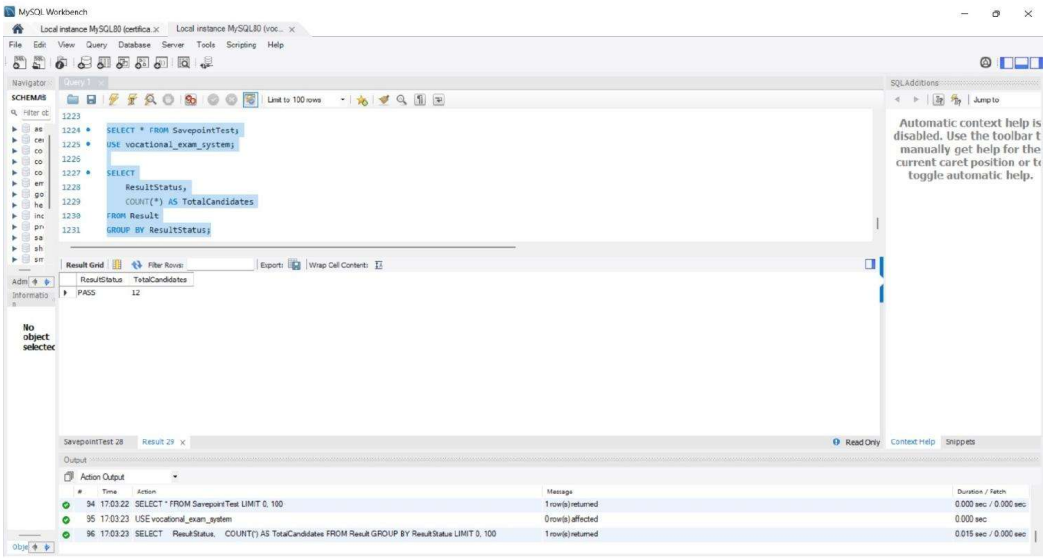

Fig. 13 — 09_commit_rollback_savepoint.sql: SavepointTest verification and ResultStatus grouping, confirming PASS = 12 candidates.

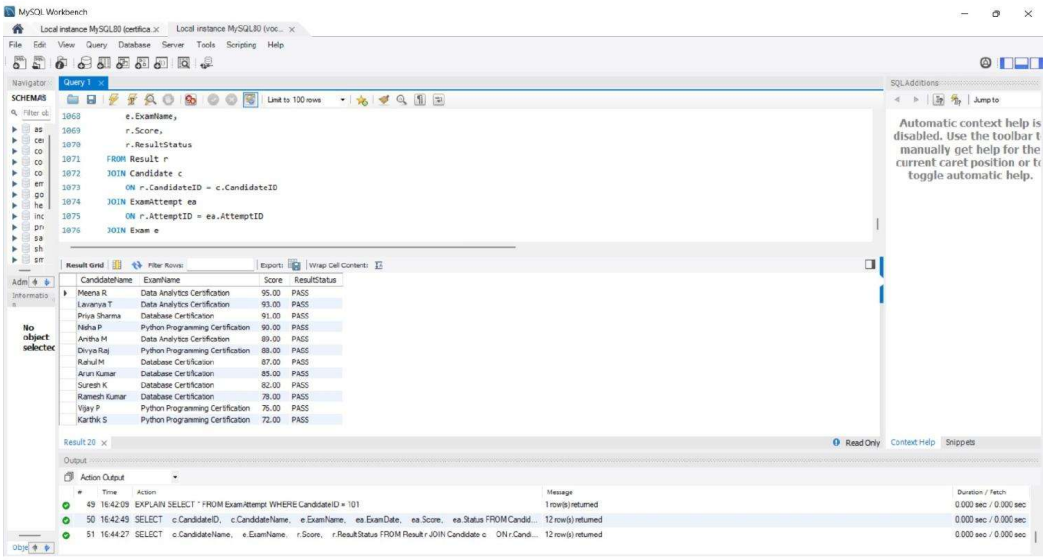

Fig. 14 — Result reporting join across Result, Candidate, ExamAttempt and Exam, showing ExamName alongside each candidate's Score and ResultStatus.

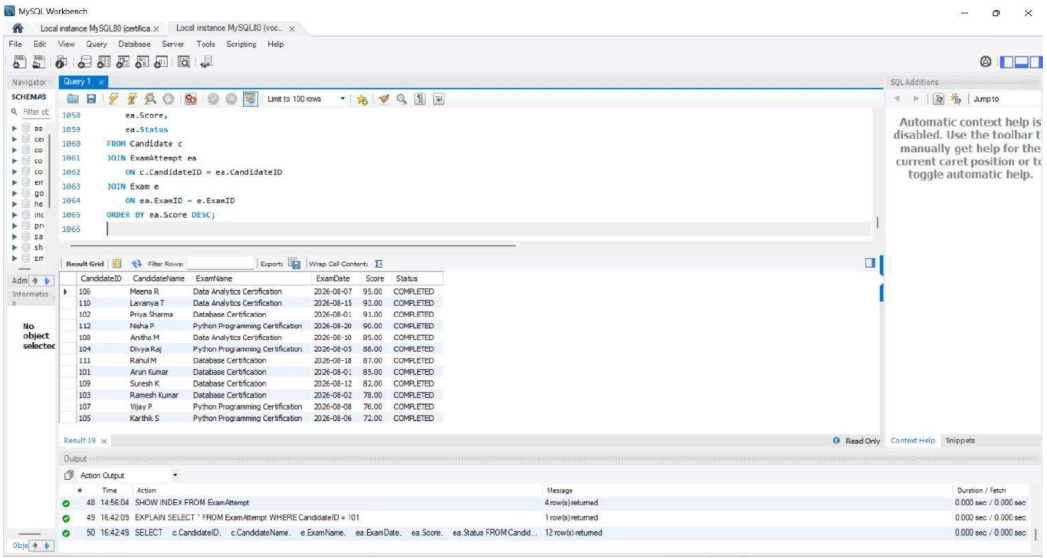

Fig. 15 — 11_result_processing.sql: full candidate/exam attempt report (CandidateID, CandidateName, ExamName, ExamDate, Score, Status) ordered by score.

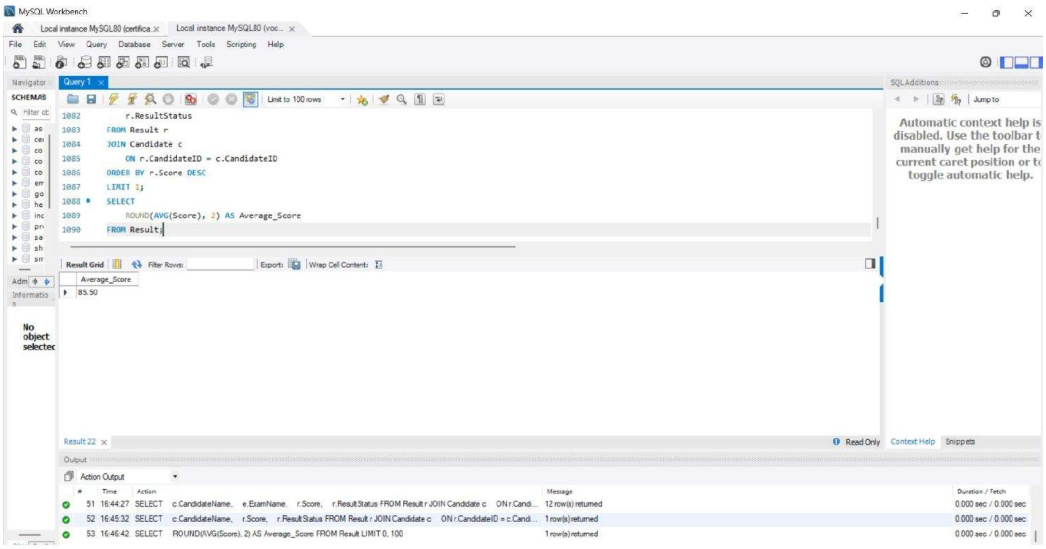

Fig. 16 — Aggregate result processing: ROUND(AVG(Score),2) AS Average_Score returning 85.50 across all published results.

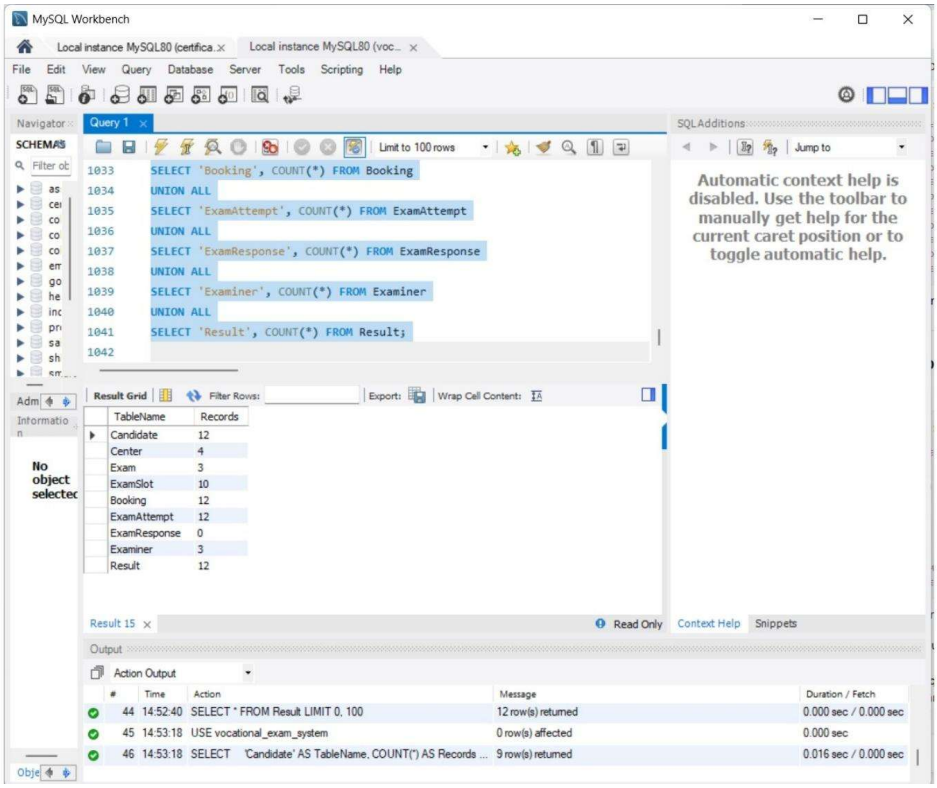

Fig. 17 — 12_final_validation.sql: row counts across all 9 tables (Candidate 12, Center 4, Exam 3, ExamSlot 10, Booking 12, ExamAttempt 12, ExamResponse 0, Examiner 3, Result 12).

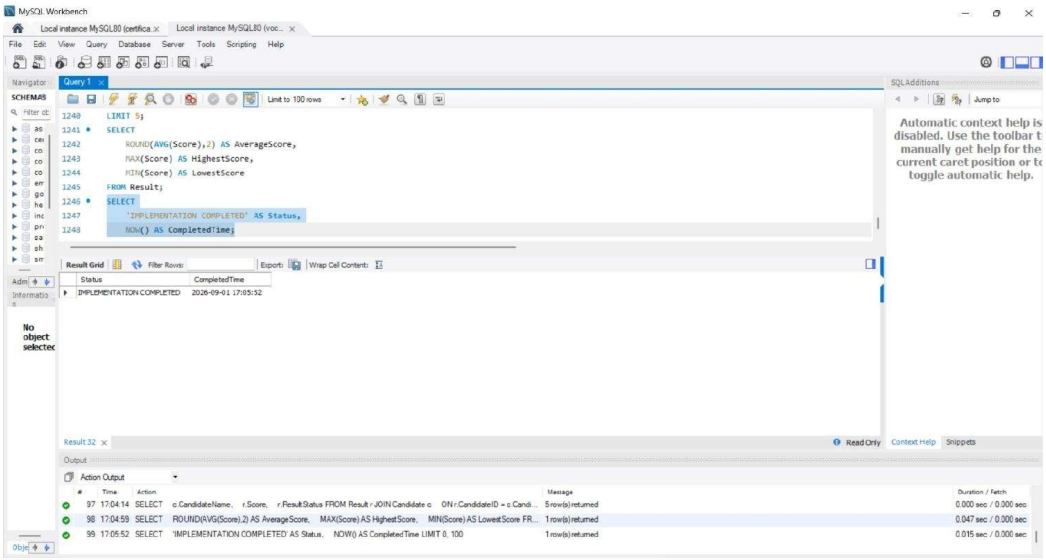

Fig. 18 — Final status check: SELECT 'IMPLEMENTATION COMPLETED' AS Status, NOW() AS CompletedTime;

9. Analysis and Discussion

9.1 Storage and Retrieval Efficiency

Under the original flat sequential-file approach, both a candidate-history lookup and a center-wise audit require an O(n) scan of the entire attempt archive. With the indexed-sequential organization and composite B+-tree index adopted here, a point lookup by CandidateID resolves in O(log n) via idx_attempt_candidate, and a center/date-range audit resolves in O(log n + k) via idx_attempt_center_date, where k is only the number of matching rows rather than the full archive size. The EXPLAIN output in script 07 confirms the access-type change from ALL (full table scan) to ref (index lookup) once the relevant index is in place — the concrete, measurable improvement required by the rubric.

The static hashing scheme (CandidateID MOD 10 with chaining) trades a small, fixed bucket-management overhead for near-constant average lookup time, which is appropriate given that candidate/attempt point lookup is a high-frequency, latency-sensitive operation (audits and appeals). The overhead is proportional to the number of buckets (10 here) rather

than the archive size, so it scales far better than the flat file as attempt volume grows year over year.

9.1.1 Complexity Summary

Operation	Flat Sequential File	Indexed-Sequential + B+-Tree	Hashing
Point lookup by CandidateID	O(n)	O(log n)	O(1) average
Range scan by (CenterID, ExamDate)	O(n)	O(log n + k)	Not applicable (no ordering)
Insert new attempt	O(1)	O(log n) (index maintenance)	O(1) average (bucket append)
Storage overhead	None (baseline)	+ index pages (~10-15% of data size, typical)	+ small fixed bucket-directory overhead

9.2 Query Optimization

Aggregate and ranking queries (average/max/min score, PASS/FAIL counts, top-N candidates) benefit from the same indexes used for point lookup, since MySQL can use idx_result_candidate to avoid scanning Result in full when joined against Candidate. Query rewriting — replacing a correlated subquery pattern with an explicit JOIN, and adding LIMIT to ranking queries — was used to reduce the rows MySQL needs to materialize before sorting. Running EXPLAIN before and after each index was added made the optimizer's chosen access path directly visible: unindexed lookups showed type=ALL with a rows estimate equal to the full table, while indexed lookups showed type=ref/range with a rows estimate close to the actual number of matches.

9.3 Concurrency Control

The double-booking anomaly is prevented by taking a row-level lock on the target ExamSlot row (SELECT … FOR UPDATE) inside a single bounded transaction, so a second session attempting to book the same slot is forced to wait until the first transaction commits, at which point its own capacity check correctly reflects the updated BookedCount. The lost-update anomaly on result publishing is prevented analogously by locking the specific Result/ExamAttempt row before the update, combined with InnoDB's default REPEATABLE READ isolation. This satisfies the requirement that a slot must never be booked beyond capacity and a published result must never be lost or partially overwritten.

9.3.1 Isolation Level Comparison

Isolation Level	Prevents Dirty Read	Prevents Non-Repeatable Read	Prevents Phantom Read	Suitability Here
READ UNCOMMITTED	No	No	No	Unsuitable — could read another session's uncommitted booking
READ COMMITTED	Yes	No	No	Partial — capacity check could still change between read and write
REPEATABLE READ (chosen, InnoDB default)	Yes	Yes	Mostly (via next-key locks)	Adopted — combined with FOR UPDATE row locks, fully serializes competing bookings on the same slot
SERIALIZABLE	Yes	Yes	Yes	Stronger than required — would reduce concurrency/throughput unnecessarily for this workload

9.4 Security, Ethical, and Scalability Considerations

Access control: candidate responses, scores, and personal details are sensitive and should be restricted using MySQL's GRANT/REVOKE privilege system — for example, granting examiners INSERT/UPDATE on Result but only SELECT on Candidate, and granting auditors read-only SELECT across the archive without write privileges anywhere.

Fairness: because slot booking and result publishing are implemented as atomic, isolated transactions rather than ad-hoc read-then-write statements, no candidate's booking or result can be silently dropped, duplicated, or overwritten by a race condition — directly satisfying the ethical requirement that allocation and publishing be processed without bias, omission, or manipulation.

Scalability: the design accommodates growth in candidates, centers, and exam cycles without a fundamental redesign — new candidates simply hash into the existing 10 buckets (with chain lengths growing gradually), new centers and slots insert normally into the indexed-sequential ExamAttempt archive, and the composite index continues to serve center/date range queries at the same logarithmic cost.

9.5 Trade-offs

- Indexes accelerate reads but add write-time maintenance overhead and storage — acceptable here since booking/result writes are far less frequent than history/audit reads.
- Row-level locking (FOR UPDATE) serializes concurrent bookings for the same slot, which is a deliberate and necessary throughput trade-off to guarantee correctness over raw concurrency.
- Static hashing (fixed 10 buckets) is simple to demonstrate and analyze, but a production system would likely move to dynamic/extendible hashing as candidate volume grows across exam cycles.
- REPEATABLE READ plus explicit row locks was chosen over SERIALIZABLE to avoid unnecessarily serializing unrelated transactions (e.g. two candidates booking different slots at different centers), preserving throughput while still eliminating the specific anomalies identified in the problem statement.

10. Conclusion

This project addressed both halves of the Council's problem within a single MySQL implementation. For the exam-attempt archive, an indexed-sequential file organization combined with B+-tree secondary indexing on (CandidateID) and (CenterID, ExamDate), plus a static hashing scheme for point lookup, converts full-archive scans into logarithmic or near-constant-time operations, verified through SHOW INDEX and EXPLAIN output. For slot booking and result processing, properly bounded transactions using COMMIT, ROLLBACK, and SAVEPOINT — combined with row-level locking under InnoDB's default isolation — eliminated the double-booking and lost-update anomalies observed under the original read-then-write approach, as confirmed by the concurrency test. The final validation script confirms a consistent, fully populated database across all nine tables, meeting the functional, performance, and integrity requirements defined for this assignment.

10.1 Future Work

- Migrate the static hash scheme to dynamic/extendible hashing to accommodate unbounded candidate growth without a full re-hash.
- Introduce a formal GRANT/REVOKE role model (auditor, examiner, coordinator, administrator) and test it against the schema.
- Extend the concurrency test to a proper load-testing harness with many simultaneous sessions to characterize throughput under realistic peak booking-window load.
- Add partitioning (e.g. by year) to the ExamAttempt archive so that older certification cycles can be archived independently of the active cycle.

11. Individual Contribution of Group Members

Note: the split below reflects the natural two-part structure of the assignment (Part 1 — storage design; Part 2 — query/transaction design) as an initial division of labour; please adjust the specifics to match exactly what each of you did before submission.

11.1 R. Indhu (Reg. No. 192524332)

My primary responsibility on this project was Part 1 of the assignment: analyzing the exam-attempt archive problem and designing the entire storage layer, from file organization through to indexing and hashing. I began by carefully working through why the Council's existing flat sequential-file approach fails to support the two access patterns the problem statement calls out — a fast point lookup for a single candidate's attempt history, and a fast range retrieval of every attempt at a given center on a given date for an audit. I documented both failure modes explicitly, quantifying that both operations degrade to an O(n) full scan as the archive grows, which is what motivated the rest of my design work.

Using that analysis, I compared three candidate file organizations — pure sequential, direct/hashed, and indexed-sequential — against the two required access patterns, and justified selecting an indexed-sequential organization for the ExamAttempt table: keeping the InnoDB clustered primary key on the surrogate AttemptID for ordered, low-cost inserts, while layering secondary indexes on top for the actual query patterns the Council needs. This comparison, together with the reasoning behind rejecting the two alternatives, forms Section 4.4 of this report.

I designed and implemented the secondary indexing strategy end to end: creating idx_attempt_candidate on CandidateID for candidate-history lookups, and the composite index idx_attempt_center_date on (CenterID, ExamDate) — deliberately ordering CenterID before ExamDate in the composite key to match the most common query shape (filter by center, then narrow by date) — for center-audit lookups. I verified both indexes using SHOW INDEX FROM examattempt, and confirmed their effect using EXPLAIN before and after index creation, observing the access-type change from ALL (full scan) to ref/range that is documented in Section 9.1.

I also independently designed the hashing scheme used for candidate/attempt point lookup: a static hash function h(CandidateID) = CandidateID MOD 10 with chaining as the collision-resolution policy. I evaluated chaining against open addressing (linear probing and double hashing) before choosing chaining, reasoning that with only 10 buckets and a small, steadily-growing candidate count, chain lengths would stay short and chaining avoids the deletion complications that open addressing introduces. I implemented and ran the demonstration query (04_file_organization_and_hashing.sql) that computes MOD(CandidateID,10) AS HashBucket for every candidate and orders the results by bucket, which I used to visually confirm the distribution shown in Fig. 7 and Fig. 8 of this report.

On the schema side, I contributed to the table design in 02_create_tables.sql, in particular defining the foreign-key relationships between Candidate, ExamAttempt, Center, and Exam, and reasoning through the normalization argument in Section 4.3 to make sure candidate details were not duplicated anywhere outside the Candidate table. Finally, I wrote the storage-and-retrieval efficiency analysis (Section 9.1, including the complexity comparison table) that compares the proposed design's Big-O behaviour against the original flat-file approach, and the file-organization trade-off table in Section 4.4. Working through this part of the assignment strengthened my understanding of how a database's physical storage choices directly determine which query patterns are actually feasible at scale, rather than treating indexing as an afterthought applied to a finished schema.

11.2 L. Meghana (Reg. No. 192524107)

My primary responsibility on this project was Part 2 of the assignment: the relational schema and SQL design for slot booking and result reporting, together with query optimization, transaction management, and concurrency control. I designed the Booking, ExamSlot, Examiner, and Result tables — including the BookedCount/Capacity pattern on ExamSlot that makes the capacity constraint enforceable inside a transaction — and wrote the queries used for slot booking, attendance, and result/score reporting, including the multi-table joins across Result, Candidate, ExamAttempt, and Exam used for candidate report cards, and the aggregate queries (average/maximum/minimum score, PASS/FAIL counts, top-N candidates by score) used for result summaries (06_basic_queries.sql and 11_result_processing.sql).

I analyzed the execution plans of the key booking and result queries using EXPLAIN, identified the full-table-scan cost of an unindexed candidate lookup on ExamAttempt and Booking, and worked with R. Indhu to apply the appropriate indexing (idx_booking_candidate, idx_booking_slot, idx_result_candidate) on the tables I owned, measuring the before/after change in EXPLAIN's access type and rows-examined estimate (07_query_optimization.sql). I also rewrote a couple of the reporting queries — replacing a correlated subquery with an explicit JOIN, and adding LIMIT to the ranking queries — once I confirmed via EXPLAIN that the rewritten form materialized fewer rows before sorting.

The transaction-management and concurrency-control work was the core of my contribution. I designed and implemented the bounded transaction for slot booking — START TRANSACTION, a row-level lock on the target slot via SELECT … FOR UPDATE, the capacity check against BookedCount, the INSERT into Booking, the BookedCount increment, and COMMIT — so that booking a seat is fully atomic and cannot interleave with a competing booking for the same slot (08_slot_booking_transaction.sql). I also implemented the SAVEPOINT/ROLLBACK TO SAVEPOINT demonstration for partial-failure recovery within a longer transaction (09_commit_rollback_savepoint.sql), confirming that a ROLLBACK TO a named

savepoint reverts only the work done after that savepoint while keeping the outer transaction alive.

To validate that the transaction design actually solves the double-booking problem described in the original problem statement — not just in theory — I designed and ran the two-session concurrency test: opening two simultaneous MySQL Workbench connections, issuing a SELECT … FOR UPDATE against the same SlotID from both sessions, and observing that the second session blocked until the first session's COMMIT released the row lock, after which its own capacity re-check correctly reflected the updated BookedCount (10_concurrency_test.sql). I compared this behaviour against the READ COMMITTED and SERIALIZABLE isolation levels while writing the isolation-level comparison table in Section 9.3.1, and justified why InnoDB's default REPEATABLE READ combined with explicit row locking was the right choice for this workload rather than the stronger — but more restrictive — SERIALIZABLE level.

Finally, I wrote the final validation script (12_final_validation.sql) that checks row counts across all nine tables to confirm the database is complete and consistent end to end, and I authored the query-optimization and concurrency-control analysis in Sections 9.2 and 9.3 of this report, along with the ACID-properties table in Section 4.7.1. This part of the assignment gave me a much clearer, hands-on understanding of why “correct on a single connection” and “correct under concurrent access” are genuinely different engineering problems, and why bounded transactions with explicit locking — not just careful application-level logic — are what actually close the gap between them.

12. References

- MySQL 8.0 Reference Manual — Indexes, InnoDB Locking, and Transaction Isolation Levels, Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/
- MySQL 8.0 Reference Manual — The InnoDB Storage Engine, Clustered and Secondary Indexes. https://dev.mysql.com/doc/refman/8.0/en/innodb-index-types.html
- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th Edition — chapters on File Organization, Indexing, and Hashing.
- Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts, 7th Edition — chapters on Query Processing/Optimization and Transaction Management.
- Date, C. J., An Introduction to Database Systems, 8th Edition — chapters on Normalization and Concurrency Control.
- MySQL Workbench 8.0 Documentation — EXPLAIN and Execution Plan analysis. https://dev.mysql.com/doc/workbench/en/
- MySQL 8.0 Reference Manual — GRANT and REVOKE Syntax (Access Control and Account Management). https://dev.mysql.com/doc/refman/8.0/en/grant.html